

1. Get oriented and set up

From tracked bee positions to activity, contacts, and nest use. BuzzAnalysis analyzes BumbleBox tracking CSVs. Each detection identifies a bee, a video frame, and its image coordinates. The tools clean those detections, display them over video, calculate behavioral measurements, and summarize changes between activity states. Video is needed for visual review; numeric analysis can run from CSVs alone.

Install once; activate each session

Use a terminal with Conda available. For a fresh installation:

```
git clone --branch testing_May25 \
  https://github.com/Crall-Lab/BuzzAnalysis.git
cd BuzzAnalysis
conda env create -f environment.yml
conda activate BuzzAnalysis
```

On August's computer, the reviewed checkout is /home/august/Desktop/new_colony_development/BuzzAnalysis. Open that folder and activate its existing BuzzAnalysis environment. The supplied environment file targets Python 3.13 and includes analysis, plotting, GUI, and test dependencies. Shapely 2 and PyArrow are needed for current geometry and binary-table exports.

Prepare one experiment

Keep originals separately and work on a copy. Store tracking CSVs with matching videos and nest annotations in a labels folder. Analysis pools one social center by default. Add --social-center-by-date-folder for separate centers under 2021-06-10, 2021_06_10, or col_01-2021-06-10-style folders, including deeper recordings. Keep a compatible colony/camera coordinate system within each group; see the README for details.

Input	What it contains
Tracking CSV	Required columns: frame, ID, centroidX, centroidY. Frames and IDs are integers; positions are pixels. frontX/frontY are optional.
Video	For GUI review, use the same recording stem as the tracking CSV, such as bumblebox-05_2026-05-01_12_00_00.mjpeg.
Nest map	LabelMe shapes converted to CSV; identifies arena/nest/foraging perimeters, brood, and food. Example: bumblebox-05-2026-05-01-nest_image.csv.

Choose the workflow

Start with the main workflow on page 2: review → split if needed → clean → analyze. It produces one summary row per bee per video and optional frame-level data.

Use the preliminary workflow on page 3 for modern *_newtracks.csv recordings and activity-transition studies. It creates a separate set of Parquet tables consumed by the new summary/plot scripts.

Older runMe*.py and analyze_data_0.1.py files remain for historical workflows. New users should start with analyze_clean_data_0.2.py. Every current command supports --help.

2. Run your first analysis

Inspect tracks and save settings

```
python review_metrics_gui.py "/path/to/working-data"
```

Select a video, expand **Metric Controls**, and choose the matching **Tracking CSV**. Use Play, the timeline, and ± 1 frame to inspect activity and contact overlays. Set Behavior FPS to the frame-number timebase of your recordings. Adjust Speed cutoff, Max speed, Inactive blip frames, Interaction px, and Nest on/off px; then click **Export settings** and save settings.json.

Defaults are 4.5 FPS, 3.5 px/frame for activity, 150 px for contacts, 12 px for nest objects, and 150 px/cm. Verify these for your experiment. Set calibration in params.py before launching the GUI. Max speed = 0 disables that filter; inactive blip frames = 0 disables smoothing. Otherwise, up to two inactive frames inside an active bout are smoothed by default.

Optional: label nest features

```
python LabelNests_GUI.1.16.py "/path/to/labels"
```

Choose an image, click **Open in LabelMe**, draw the relevant labeled shapes, and save. Return to LabelNests and choose **Convert all JSON files in folder to CSV**. This overwrites existing generated CSVs of the same name. Nest-map device/colony number and date must match the tracking filename.

Clean a copied recording

The following small example uses the CSV included in the repository. Run these commands from the repository folder. For your own files, replace the sample with copied recordings and select their exact raw suffix.

```
mkdir -p demo/raw
cp worker23_2022-06-20_18-25-30.csv demo/raw/
python 01_split_lr.py -s demo/raw -e .csv --whole
python 02_clean_data.py -s demo/raw
```

--whole keeps the whole image; omit it only for a two-arena recording and set --threshold to its dividing X coordinate. Splitting creates per-recording folders. Cleaning creates *_clean.csv files beside the split files. It resolves duplicate IDs; unresolved duplicates are dropped. Add --settings settings.json to apply a nonzero maximum-speed cutoff to isolated spike removal.

Interpolation is optional: add --interpolate --real-fps 4.5 --max-interp-sec 3, replacing those values with your validated timing. Filled rows have interpolated=1; original rows have 0. Ambiguous one-way jumps are retained and logged.

Calculate summaries

```
python analyze_clean_data_0.2.py -s demo/raw \
-o demo/Analysis.csv --save-frame-level
```

For your experiment, add --settings settings.json and optionally -b "/path/to/labels". Explicit command-line options override JSON settings, which override defaults. Use -l 1 for a first-file check; -c 4 enables four workers. Supported names start with worker, col, colony, or bumblebox and contain date/time; custom layouts require --filename-positions and the inclusive positions in params.py.

3. Understand and extend results

Output	Contents
Analysis.csv	One bee/video/side per row: activity, speed, coordinates, contact totals, and optional brood metrics.
*_frame_level.csv	One bee/frame per row: positions, activity, contacts, distances, on-object flags, and location.
*_pivot_enriched.feather	Optional wide coordinate/distance/activity/speed tables; enable --save-pivots, read with pandas/PyArrow.

Units: activity is 1 (active), 0 (inactive), or missing (unscored). First observations, tracking gaps, and rejected speeds can be unscored. Speed is px/frame; multiply by FPS ÷ pixels/cm for cm/s. Nest and social-center distances are pixels; meanIBD and medianMinDistToOthers are centimeters. totalInt counts close bee-pair frames, not interaction bouts. totalIntFrames counts observed pair opportunities; it can exceed video frame count. Location prioritizes nest, then foraging, then arena. Missing maps give unknown location.

Study activity transitions

This workflow expects bumblebox-05_2026-05-01_12_00_00_newtracks.csv-style names. Replace paths/device/colony below; use a fresh results directory.

```
python preliminary_newtracks_analysis.py \
  --source /path/to/tracks --labels /path/to/labels \
  --output results --box-id bumblebox-05 --colony-number 65 \
  --settings settings.json --no-infer-foraging
python summarize_inactive_to_active.py results
python summarize_nest_transition_rates.py results
```

This writes daily frame/contact Parquet tables, CSV indexes, summaries, and run_metadata.json. Raw duplicate coordinates are averaged; this preliminary path does not replace cleaning. --no-infer-foraging uses explicit labels rather than the optional rightmost-arena heuristic. Use --exclude-bee-ids 7 23 for problematic tags; exclusions carry into summaries and contact recalculation.

```
python plot_activity_transition_rates.py \
  results/activity_transition_rates_by_location_contact.csv
```

Plots write PNG/PDF. plot_inactive_to_active_over_time.py plots a date summary; set --fps for your experiment. Transition rates use adjacent scored frames within each bee/video, grouped by starting state and context. Rate/second = rate/frame × FPS, not a one-second probability.

Troubleshoot and check

No files: check source and suffix (-e). **Missing/ambiguous map:** keep one matching device/date map. **Unexpected activity:** check FPS, calibration, smoothing, and video. **Blank metrics:** inspect missing detections and terminal warnings. Test the software with QT_QPA_PLATFORM=offscreen python -m pytest -q. Preserve settings with results and inspect representative videos before scaling up.